

SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA1735 – ARTIFICIAL INTELLIGENCE

SLOT A

LAB EXPERIMENTS

KAUSHIK - 192321047

KAUSHIK NARAYANAN V - 192321047

FACULTY

Dr. Thalapathi Rajasekaran R

Experiment 6: Vacuum Cleaner Problem

Aim

To write a Python program to solve the Vacuum Cleaner problem using a simple rule-based approach.

Algorithm

1. Initialize two rooms, A and B, as either dirty or clean.
2. Place the vacuum cleaner in room A.
3. Check whether the current room is dirty.
4. If dirty, clean the room.
5. Move to the next room and repeat the cleaning process.
6. Display the final status of both rooms.

Program

```
rooms = {"A": "Dirty", "B": "Dirty"}

position = "A"
```

KAUSHIK - 192321047

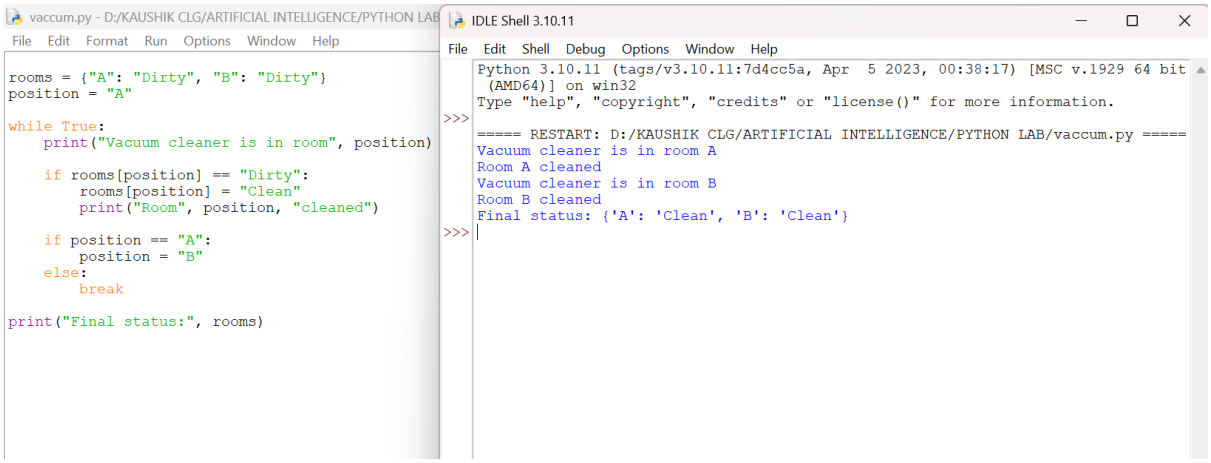
```
while True:
    print("Vacuum cleaner is in room", position)


    if rooms[position] == "Dirty":
        rooms[position] = "Clean"
        print("Room", position, "cleaned")


    if position == "A":
        position = "B"
    else:
        break


print("Final status:", rooms)
```

Output



Result: The Vacuum Cleaner problem is solved successfully.

KAUSHIK - 192321047

Experiment 7: Breadth-First Search (BFS)

Aim

To write a Python program to implement the Breadth-First Search algorithm for graph traversal.

Algorithm

1. Start from the initial node.
2. Create a queue and insert the starting node.
3. Mark the starting node as visited.
4. Remove the first node from the queue and display it.
5. Add all unvisited adjacent nodes to the queue.
6. Repeat until the queue becomes empty.
7. Display the traversal order.

Program

```
from collections import deque
```

KAUSHIK - 192321047

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

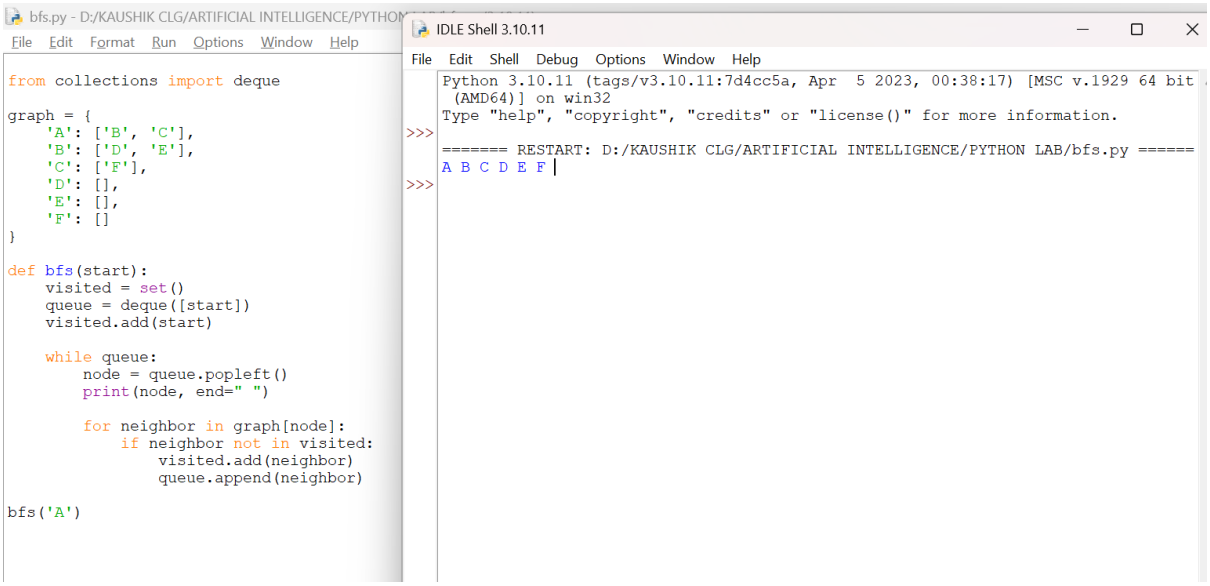

def bfs(start):
    visited = set()
    queue = deque([start])
    visited.add(start)
```

```
    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

bfs('A')
```

Output



Result: BFS is implemented successfully using a queue.

Experiment 8: Depth-First Search (DFS)

Aim

To write a Python program to implement the Depth-First Search algorithm for graph traversal.

Algorithm

1. Start from the initial node.
2. Mark the node as visited.
3. Display the node.
4. Visit an unvisited adjacent node recursively.
5. Continue until no unvisited adjacent nodes remain.
6. Backtrack to the previous node.
7. Repeat until all reachable nodes are visited.

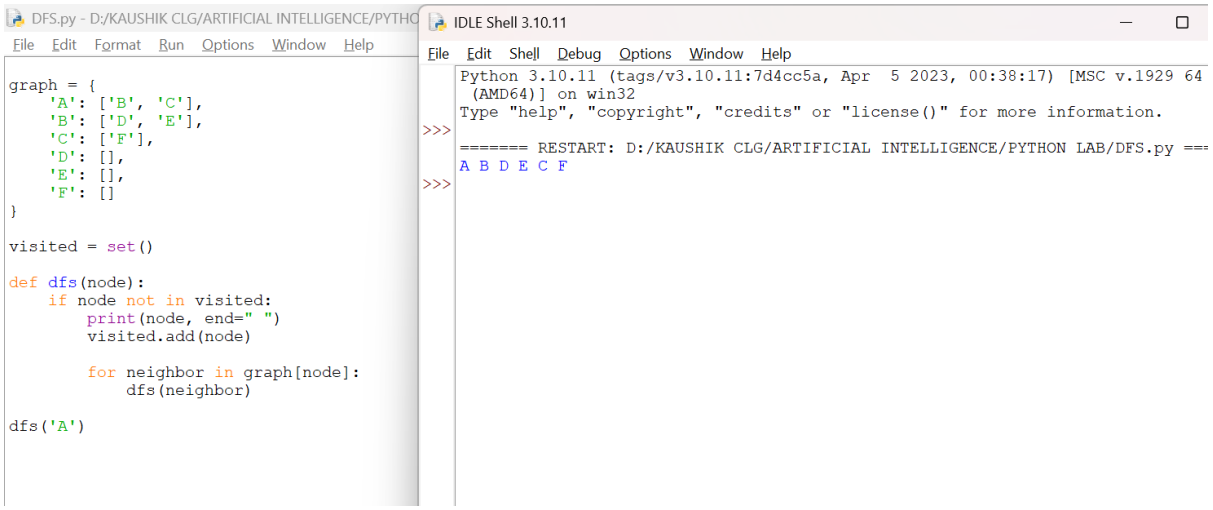
Program

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
visited = set()
def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)
```

KAUSHIK - 192321047

dfs('A')

Output



Result: DFS is implemented successfully using recursion.

KAUSHIK - 192321047

Experiment 9: Travelling Salesman Problem (TSP)

Aim

To write a Python program to solve the Travelling Salesman Problem by finding the minimum-cost tour.

Algorithm

1. Represent the cities and distances using a cost matrix.
2. Select the first city as the starting city.
3. Generate all possible orders of the remaining cities.
4. Calculate the total travel cost for each tour, including the return to the starting city.
5. Compare the costs of all tours.
6. Display the tour with the minimum cost and its total distance.

Program

```
from itertools import permutations
cost = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
n = len(cost)
cities = list(range(1, n))
min_cost = float('inf')
best_path = []

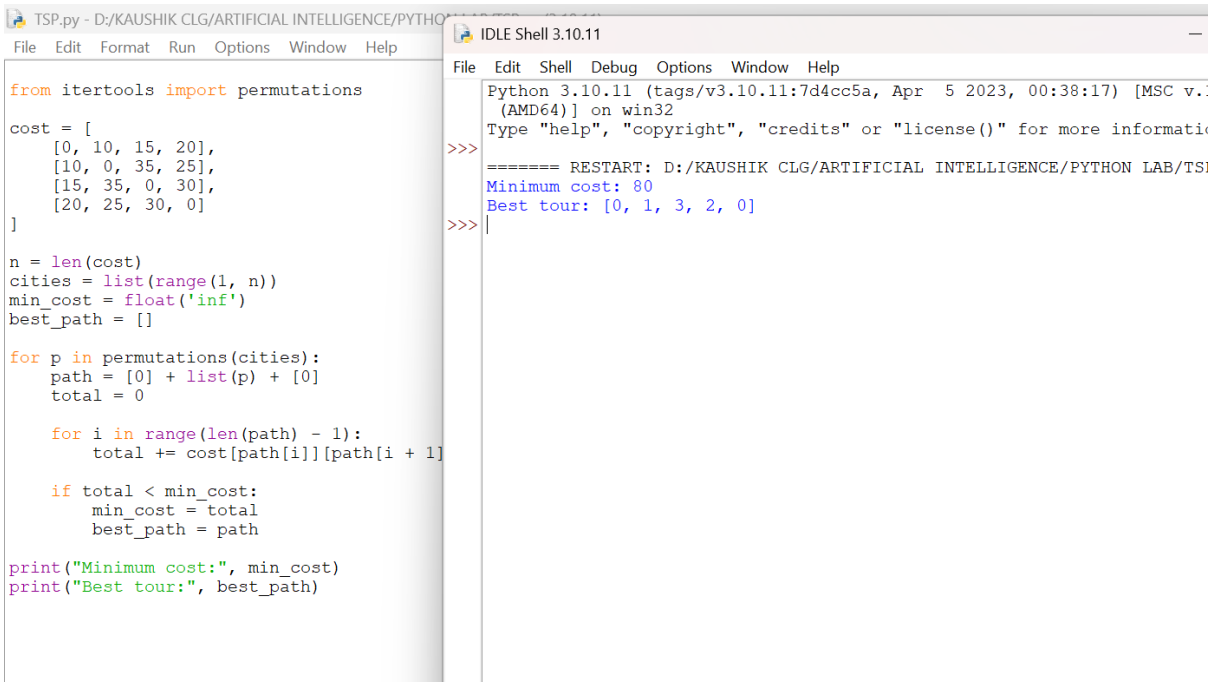
for p in permutations(cities):
    path = [0] + list(p) + [0]
    total = 0
```

KAUSHIK - 192321047

```
    for i in range(len(path) - 1):
        total += cost[path[i]][path[i + 1]]

    if total < min_cost:
        min_cost = total
        best_path = path
print("Minimum cost:", min_cost)
print("Best tour:", best_path)
```

Output



Result: The Travelling Salesman Problem is solved successfully by finding the minimum-cost tour.

Experiment 10: A* Algorithm

Aim

To write a Python program to implement the A* search algorithm to find the shortest path in a graph using path cost and heuristic values.

Algorithm

1. Initialize the starting node and goal node.
2. Assign a path cost \(g(n)\) and heuristic value \(h(n)\) to each node.
3. Calculate the evaluation function \(f(n)=g(n)+h(n)\).
4. Select the node with the lowest evaluation value.
5. Explore its neighboring nodes and calculate their new path costs.
6. Update the costs and paths when a better route is found.
7. Repeat until the goal node is reached.
8. Display the shortest path and its total cost.

Program

KAUSHIK - 192321047

```
import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1)],
    'D': []
}

h = {'A': 4, 'B': 3, 'C': 1, 'D': 0}

def astar(start, goal):
    queue = [(h[start], 0, start, [start])]
    best = {start: 0}
```

```
    while queue:
        f, g, node, path = heapq.heappop(queue)

        if node == goal:
            print("Path:", path)
            print("Cost:", g)
            return

        if g > best[node]:
            continue

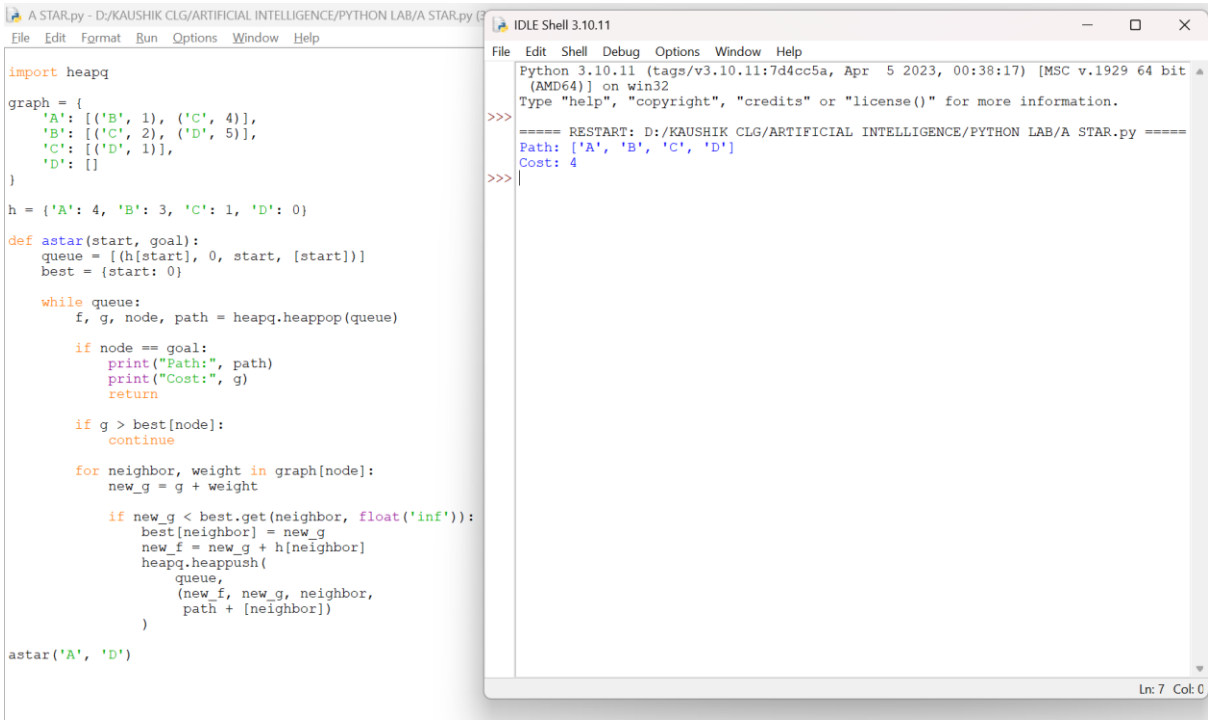
        for neighbor, weight in graph[node]:
            new_g = g + weight

            if new_g < best.get(neighbor, float('inf')):
                best[neighbor] = new_g
                new_f = new_g + h[neighbor]
                heapq.heappush(
                    queue,
                    (new_f, new_g, neighbor,
                     path + [neighbor])
                )

astar('A', 'D')
```

KAUSHIK - 192321047

Output



```
A STAR.py - D:/KAUSHIK CLG/ARTIFICIAL INTELLIGENCE/PYTHON LAB/A STAR.py (3
File  Edit  Format  Run  Options  Window  Help

import heapq

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('C', 2), ('D', 5)],
    'C': [('D', 1)],
    'D': []
}

h = {'A': 4, 'B': 3, 'C': 1, 'D': 0}

def astar(start, goal):
    queue = [(h[start], 0, start, [start])]
    best = {start: 0}

    while queue:
        f, g, node, path = heapq.heappop(queue)

        if node == goal:
            print("Path:", path)
            print("Cost:", g)
            return

        if g > best[node]:
            continue

        for neighbor, weight in graph[node]:
            new_g = g + weight

            if new_g < best.get(neighbor, float('inf')):
                best[neighbor] = new_g
                new_f = new_g + h[neighbor]
                heapq.heappush(
                    queue,
                    (new_f, new_g, neighbor,
                     path + [neighbor])
                )

astar('A', 'D')
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:/KAUSHIK CLG/ARTIFICIAL INTELLIGENCE/PYTHON LAB/A STAR.py =====
Path: ['A', 'B', 'C', 'D']
Cost: 4
>>>
Ln: 7  Col: 0
```

KAUSHIK - 192321047

Result: The A* algorithm is implemented successfully to find the shortest path using heuristic search.